

A Spatial Data Structure for Fast Poisson-Disk Sample Generation

Daniel Dunbar
University of Virginia

Greg Humphreys
University of Virginia

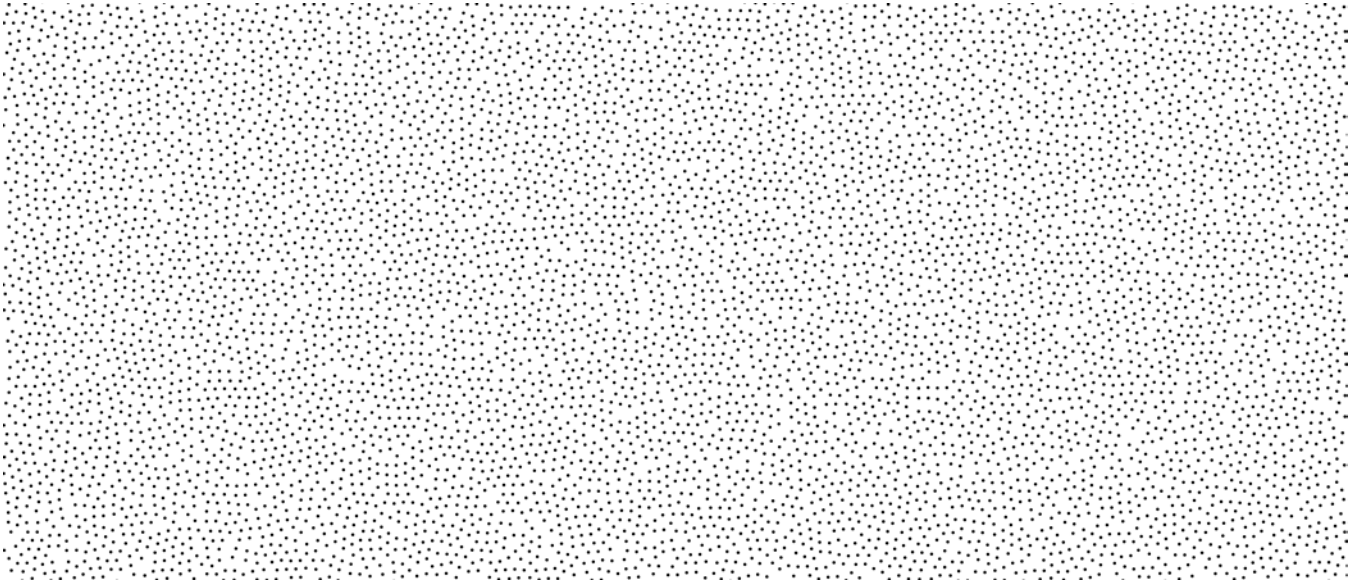


Figure 1: A Poisson-disk distribution of 17,593 points generated in 80 ms.

Abstract

Sampling distributions with blue noise characteristics are widely used in computer graphics. Although Poisson-disk distributions are known to have excellent blue noise characteristics, they are generally regarded as too computationally expensive to generate in real time. We present a new method for sampling by dart-throwing in $O(N \log N)$ time and introduce a novel and efficient variation for generating Poisson-disk distributions in $O(N)$ time and space.

CR Categories: I.3.3 [Computer Graphics]: Picture/Image Generation—Antialiasing; I.4.1 [Image Processing and Computer Vision]: Digitization and Image Capture—Sampling

Keywords: sampling, blue noise, poisson disk

1 Introduction and Background

Almost all problems in computer graphics involve sampling. It is well known that the properties of the sampling distribution can greatly affect the quality of the final result. In

particular, blue-noise patterns perform especially well in this setting because of the low-energy annulus around the DC spike in their frequency spectrum. High quality sampling patterns are especially important when sampling the image plane in a raytracer, not only because they do a better job of capturing the continuous function being sampled, but also because in this setting the function being reconstructed is displayed directly, so any sampling errors will be especially apparent to a viewer.

Poisson-disk distributions have excellent blue noise spectra and also mimic the distribution of photoreceptors in a primate eye [Yellot 1983]. These distributions have proven difficult to generate directly, so many alternate approaches have been developed, few of which can guarantee the Poisson-disk property. In this paper, we describe an $O(N \log N)$ algorithm for directly generating maximal Poisson-disk distributions identical to those produced by a dart-throwing technique. We then present a variation of this algorithm that both yields better spectral distributions and runs in linear time and space. This algorithm generates point sets with excellent blue noise characteristics very quickly; it can generate over 200,000 points per second on a modern CPU.

1.1 Previous Work

Sampling theory is a well researched area of research in computer graphics, and it has even deeper roots in the signal processing and information theory literature. Stochastic sampling was first introduced to computer graphics by Dippé and Wold [1985]. Cook analyzed the spectral properties of various stochastic point processes [Cook 1986]. In that paper, he extols the virtues of Poisson-disk distributions be-

cause of their blue noise properties and relationship to photoreceptor distributions, but ultimately advocates the use of jittered grids because the straightforward dart-throwing algorithm for generating Poisson-disk distributions is prohibitively expensive. Since then, many algorithms have been proposed for generating point distributions; e.g. [Ulichney 1988; Shirley 1991; Hiller et al. 2001; Kollig and Keller 2002; Kollig and Keller 2003].

Mitchell’s $O(N^2)$ “best-candidate” algorithm attempts to mimic dart-throwing while providing a termination guarantee [Mitchell 1991]. Whenever a new sample is to be drawn, a number of candidate samples are randomly generated, and the candidate that is farthest from the existing point set is accepted. This algorithm cannot guarantee the Poisson-disk property, but in practice it generates excellent point sets if enough candidates are drawn. The primary drawback of this technique is its long running time.

McCool and Fiume generated high-quality tile sets with a toroidal distance function so that they could be repeatedly tiled across the plane [McCool and Fiume 1992] and introduced the use of Lloyd’s relaxation to improve the blue noise properties of the set. Lloyd’s relaxation transforms a point set by moving each point to the center of its associated Voronoi region and is typically applied iteratively or used to generate distributions directly [Hiller et al. 2001]. Once the tiles have been generated, this method generates large point sets very efficiently. However, the tiling process introduces easily recognizable structures. This is acceptable for most sampling applications as long as the tile size is large enough, but is disastrous if the points are being used to distribute objects or as part of a texture basis function. Several schemes have been proposed to solve this problem.

Multiple authors have proposed using Wang tiles to solve this problem. Cohen et al. populated a set of Wang tiles with small point distributions that were intended to tile the plane [Cohen et al. 2003]. Lagae et al. showed that correctly applying this technique requires careful attention to how points are placed at the boundary of a tile and presented *Poisson-disk tiles* to address these issues [Lagae and Dutré 2005]. Most recently, Kopf et al. have extended these techniques to allow generation of point sets with blue noise properties satisfying an arbitrary density function [Kopf et al. 2006]. Their technique produces high quality point sets very efficiently once the tile set has been computed.

Ostromoukhov et al. described a fast technique for importance sampling a provided density function [Ostromoukhov et al. 2004]. Their algorithm generates point sets with local blue noise characteristics by using a clever modification of Penrose tiles and exploiting the tilings’ aperiodic nature. However, when used to generate many points from a constant density function, the resulting point sets have larger angular anisotropy than techniques based on randomness.

Jones presents an algorithm for generating 2D Poisson-disk distributions in $O(N \log N)$ time [Jones 2006]. Like our technique, Jones’ method builds a point set incrementally by storing neighboring regions of points in a balanced tree and inserting points into these regions one by one. However, neighbor regions are represented as Voronoi cells and Jones’ method requires an incremental Delaunay triangulation algorithm with $O(\log N)$ performance when adding a point. This makes the implementation more complicated than that of our algorithm. Furthermore, in this paper we show a variation of our algorithm that runs in linear time.

2 Dart-Throwing in $O(N \log N)$

The dart-throwing method for computing Poisson-disk distributions iteratively refines an existing point set by generating a series of random candidate points in the sample domain and keeping only the first such point that is farther than the minimum distance $2r$ from all other points. Each sample effectively invalidates a disk of radius $2r$ centered around that point.

This algorithm is simple to implement and extends naturally to any domain with a well-defined and computable distance metric. However, the algorithm may not terminate, so in practice the algorithm is stopped after some fixed number of consecutive candidates have failed to be accepted.

One side effect of this approximation is that the generated point set is usually not maximal (there may be regions where a point could be placed without violating the distance criterion), so some regions of the domain may be undersampled. This problem is usually ignored, although Jones’ technique is guaranteed to generate maximal distributions [Jones 2006]. Furthermore, since a large number of sample points is typically required, the algorithm is too slow to use directly. Consequently, several schemes for precomputing small distributions and tiling them have been proposed [Hiller et al. 2001; Cohen et al. 2003; Lagae and Dutré 2005].

We call the subdomain within which it is legal to add a point the *available subdomain*. Let $D(x, r)$ be the disk of radius r around a point x . For a domain X and existing point set P , the available subdomain is given by

$$A_X = X - \bigcup_{p \in P} D(p, 2r).$$

The key to emulating dart-throwing efficiently is the observation that we do not need to sample the entire available subdomain. Consider the annulus between radii $2r$ and $4r$ around some point. Every point in this annulus must be unavailable in any maximal distribution and therefore within a distance of $2r$ from some other point. This means that there must be at least one point that lies inside the annulus, and hence the union of all such annuli, intersected with the available subdomain, must contain at least one point. Therefore, it is possible to emulate dart-throwing by sampling from only this region, which we call the *available neighborhood* of a point set P . By carefully choosing the representation for this region, dart-throwing can be implemented in $O(N \log N)$ time.

2.1 Representing the Available Neighborhood

The problem of representing the available neighborhood can be divided into two parts: developing a spatial structure for the available region of the annulus, and partitioning the available neighborhood so that these structures can be efficiently updated upon the insertion of a new point. In addition, it must be possible to quickly generate a uniformly distributed random point inside the region.

Our solution to the first part of this problem involves a new data structure, the *scaloped region*. This data structure can be used to efficiently represent arbitrary boolean operations on 2D disks. We divide scaloped regions into a disjoint union of *scaloped sectors* (Figure 2). A scaloped sector

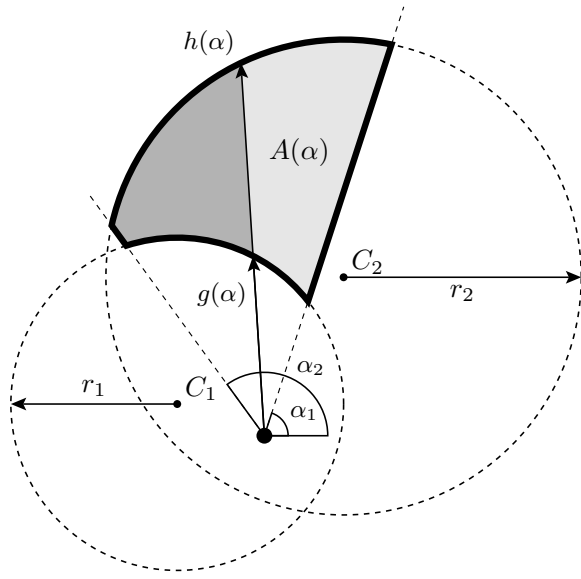


Figure 2: A *scalloped sector* is a sector bounded above and below by circular arcs. In the above diagram, α ranges from α_1 to α_2 , and $g(\alpha)$ and $h(\alpha)$ are the distance functions from the sector's apex to the near and far arcs, respectively. $A(\alpha)$ is the partial area of the sector up to α , represented by the light gray area in the figure.

centered at the origin is defined as the sector lying between angles α_1 and α_2 and bounded by near and far circular arcs. For convenience, the circular arcs are required to be non-intersecting, although they are allowed to meet at the edge of the sector.

The circular arcs are each described by a center C , radius r , and a sign $k \in \{-1, 1\}$ which selects either the near or far arc of the circle. The distance to this circular arc along a ray at angle α is then given by

$$d \cos(\alpha - \gamma) + k \sqrt{r^2 - (d \sin(\alpha - \gamma))^2},$$

where (d, γ) are the polar coordinates of C (taking the apex of the sector to be the origin). If $g(\alpha)$ and $h(\alpha)$ represent the distance functions to the inner and outer bounding arcs respectively then the area of the sector up to an angle α is given by

$$A(\alpha) = \int_{\alpha_1}^{\alpha} \int_{g(\theta)}^{h(\theta)} r \, dr \, d\theta,$$

and the area of the full sector is $A(\alpha_2)$.

The probability of a random point in a scalloped region falling within a particular sector is the area of the sector divided by the area of the region. A uniform random point within the region can be generated by randomly choosing a sector using the sectors' areas as a probability distribution and then generating a uniformly distributed point inside the chosen sector. Generating the point inside the sector is done by transforming a uniformly distributed random point (ξ_1, ξ_2) in $[0, 1]^2$ to a point (d, θ) in polar coordinates, where

$$\begin{aligned} d &= \sqrt{g(\theta)^2 + \xi_1(h(\theta)^2 - g(\theta)^2)} \\ \theta &= A^{-1}(\xi_2 A(\alpha_2)). \end{aligned}$$

Although we do not have a closed form equation for A^{-1} , A is the integral of a function that is both non-negative and zero only at the angular endpoints, so A is monotonically increasing and therefore invertible. In practice binary search is sufficient to evaluate A^{-1} efficiently.

The result of a boolean union, intersection, or difference operation between a single scalloped sector and a disk can always be subdivided into a small number¹ of new scalloped sectors. A scalloped region maintains a list of its constituent scalloped sectors and operations are performed by replacing each sector with the result of applying the operation to each individual sector. A complete description of how to locate the new scalloped sectors requires enumerating a number of cases and is omitted here for brevity; the full details can be found in our extended technical report [Dunbar and Humphreys 2006].

The available neighborhood is partitioned into scalloped sectors of outer radius $4r$ around each point in order to restrict the number of sectors that must be updated after point insertion to a small constant. Efficient sampling, however, requires that all of these neighborhoods be disjoint. In general, if an ordering relation is defined for a set S of sets it is possible to derive a new set S' of *disjoint* sets where

$$\bigcup_{s' \in S'} s' = \bigcup_{s \in S} s,$$

by subtracting from each set the union of all members of S that are less than it in the relation.

We use the generation order of the points as an ordering relation and then define the available neighborhood of a point $p \in P$ as

$$N_p = D(p, 4r) - \bigcup_{p' \in P} \begin{cases} D(p', 4r), & p' < p \\ D(p', 2r), & p' \geq p \end{cases}.$$

The available neighborhood is $N = \bigcup_{p \in P} N_p$ (Figure 3). Each disjoint N_p is computed using boolean disk subtraction.

2.2 Algorithm Details and Complexity Analysis

Our algorithm A_1 for efficient dart-throwing begins with an initial set consisting of a single point randomly chosen in the domain. During sample generation, we maintain an associative map from candidate points (points with non-empty available neighborhoods) to their associated neighborhoods.

A candidate point is then randomly chosen (using neighborhood areas as a probability distribution) and a random point within its neighborhood is added to the point set. The available neighborhood for the new point is an annulus from radii $2r$ to $4r$, minus a disk of radius $4r$ around the nearby points. The maximum distance required to search for neighbors is $8r$ since the scalloped region and neighbor disk are both bounded by $4r$. All nearby neighborhoods are then updated by subtracting a disk of radius $2r$ around the newly inserted point. This process continues until no candidate points remain.

The maximum number of scalloped sectors in an available neighborhood is bounded by a constant. Furthermore, the

¹Bounded by a constant that depends only on the particular operation (e.g., seven for disk subtraction).

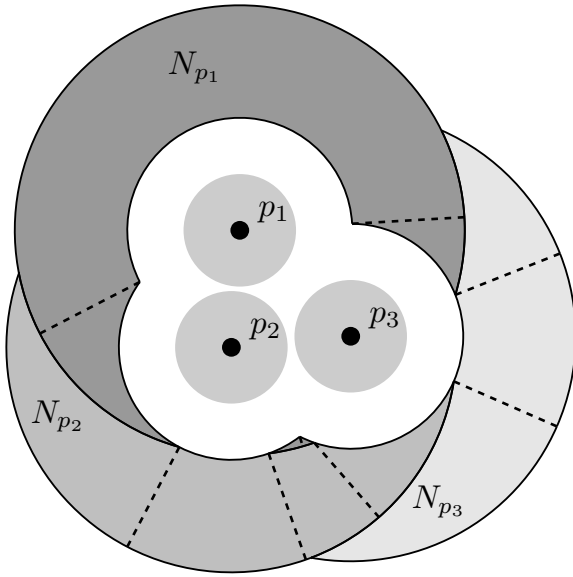


Figure 3: A partial point set and its neighborhoods. The dashed lines represent scalloped sector boundaries within a region.

Poisson-disk distance condition bounds the number of neighbors within a fixed radius. We can therefore use a uniform grid to implement the neighbor search and update of the available neighborhoods in $O(1)$ time. Similarly, picking an individual scalloped sector within an available neighborhood and generating a point in that sector can be done in $O(1)$ time. By storing the available neighborhoods in a balanced tree structure, we can choose an available neighborhood and update the tree within $O(\log N)$ time, so the time complexity of the entire algorithm is $O(N \log N)$ where N is the number of generated points. The space complexity is $O(N)$.

If we drop the requirement that the available neighborhoods be sampled according to an area-weighted probability density function then this new algorithm A_2 runs in linear time. This is a significant theoretical speedup, but in practice the cost of maintaining the sectors, intersecting them with disks, and updating data structures dominates the running time.

3 Boundary Sampling

In this section, we show how the algorithms described in the previous section can be modified to avoid the complexity of sector operations, thereby generating Poisson-disk point sets in linear time extremely quickly. In particular, we generalize either A_1 or A_2 by making the outer radius of the annulus defining the available neighborhoods a parameter r' , where $2r < r' \leq 4r$. This modification does not change the structure or performance of either algorithm, and we will assume that they are parameterized by r' for the rest of the paper.

With this change, the available neighborhood is defined as

$$N_p = D(p, r') - \bigcup_{p' \in P} \begin{cases} D(p', r'), & p' < p \\ D(p', 2r), & p' \geq p \end{cases}.$$

Notice that the overall density of the generated point set will tend to be inversely proportional to r' ; this can be exploited

for applications such as randomized object placement, in which it is desirable to tune the density of the point set.

A special case arises if r' is taken to be the minimum value $2r$. In this case, a point's available neighborhood collapses to a collection of circular arcs centered at the point. We call these arcs the *available boundary*. By directly implementing boundary sampling, we no longer need to represent the available neighborhood as scalloped regions; instead, the available boundary is represented as a set of per-point angular ranges at which a point can be placed on the boundary.

Additionally, if we select the new candidate point at random instead of according to the length of its available boundary (similarly to how we obtained the linear algorithm A_2), it is no longer necessary to explicitly store the neighborhoods for every point already in the set. Once a candidate has been chosen, its available boundary can be quickly computed by intersecting the boundary circles of the candidate with its immediate neighbors. After the legal ranges have been determined, we can repeatedly place new points at available locations on its boundary until the available boundary is empty. The addition of a new point only requires subtracting a single angular range from the candidate's boundary.

The resulting algorithm A_3 , which we call *boundary sampling*, is simple to implement and runs in $O(N)$ time and space (pseudo code is given in our technical report [Dunbar and Humphreys 2006] and an implementation is available from our website). Our implementation is approximately 200 lines of C++ code and can generate over 200,000 points per second on a 3 GHz Pentium 4. Figure 1 shows an example point set generated using our algorithm.

4 Results

In this section, we show results from the boundary sampling algorithm described in Section 3, and compare them to other methods for computing Poisson-disk distributions.

The best tiling schemes can generate point sets very efficiently and with spectra comparable to dart-throwing or Lloyd's relaxation, although there will be energy and anisotropy spikes associated with any tiling. Although we do not currently have access to an implementation of a sophisticated tiling method, we expect that the runtime performance of our algorithm is comparable to that of a tiling scheme, but our results will be artifact-free and require no precomputation.

We analyze the properties of two-dimensional noise distributions in the style of McCool and Fiume, who compute the radial power and anisotropy using the periodogram of a point set [McCool and Fiume 1992]. The primary characteristic of a blue noise distribution is a low energy annulus around the central DC spike with energy returning to a relatively constant value outside the annulus. The quality of a distribution depends on the magnitude of the difference between the DC spike, the low energy annulus, and the average energy in the high frequencies. Evaluating the distribution in terms of radial power also requires analyzing the radial anisotropy to ensure that the radial power spectrum is an accurate representation of the pattern along all orientations.

Figure 4 shows the radial power spectra of boundary sampling compared to both dart-throwing and linearized dart-throwing. The graph shows that neither linearized sampling

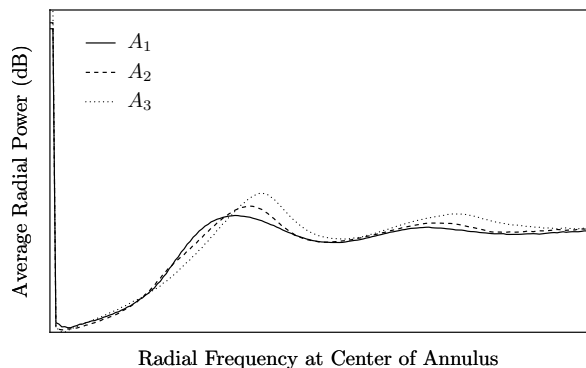


Figure 4: The blue noise properties of the distribution are preserved even if only the boundary of the available neighborhood is sampled. Sampling without regard to the probability density of available neighborhoods also preserves the blue noise properties.

nor boundary sampling significantly change the blue noise properties of the resulting distributions.

Figure 5 displays averaged periodograms for 100 point sets having a radius of 0.02, resulting in approximately 2000 points each. The boundary sampling periodogram shows that the higher point density results in a greater magnitude difference between the low energy annulus and the peak transition energy. The periodograms also show that the method of Ostromoukhov et al. is significantly less uniform than that of our method. They address this issue by precomputing relaxation vectors, but these precomputed tables are only sufficient to improve the blue noise properties of small local regions of the generated distribution. As the number of points grows, the lookup table is no longer able to compensate for the inherent structure of the Penrose tiling. Because we are interested in using these point sets for sampling the image plane in high quality image synthesis, hundreds of millions of points will likely be required, and the lookup tables would become quite large.

Timing results for several methods of computing Poisson-disk distributions are shown in Table 1. For methods that require specification of a radius, one is chosen so that the number of generated points is approximately equal to the given value of N , and the time is computed as N times the average number of points per second. The times for Ostromoukhov’s method were generated with code provided by the authors of that paper, although they state that the provided code is not fully optimized. The results show that although the linear approximation of dart-throwing (A_2) is more efficient than true dart-throwing (A_1) for large numbers of points, the computational overhead of sector subtraction still adds significant overhead compared to boundary sampling (A_3). For small numbers of points, A_2 may perform more disk subtractions than A_1 , making it less efficient than its $O(N \log N)$ counterpart.

5 Conclusion and Future Work

We have described a new technique for efficiently implementing the dart-throwing algorithm for the generation of Poisson-disk point sets, based on the manipulation of disjoint unions of scalloped sectors. This algorithm runs in

N	1000	10000	100000
Best Candidate	1.454	157.014	6.084h
Dart-throwing (A_1)	0.573	5.905	141.901
$O(N)$ Dart-throwing (A_2)	0.667	6.442	61.186
Ostromoukhov et al.	0.015	0.095	1.546
Boundary Sampling (A_3)	0.001	.058	0.496

Table 1: Timing results for generating point sets of varying sizes. Times are in seconds except where otherwise noted.

$O(N \log N)$ time and motivates a new algorithm for generating 2D Poisson-disk point sets that runs in $O(N)$ time and space and produces excellent blue noise patterns. Here we present three related avenues for future research.

5.1 Sampling on Arbitrary Manifolds

The boundary sampling method can be extended to sample over any manifold with an associated distance metric. All that is required is an efficient method for finding the local neighbors of a point and for determining the intersection of adjacent boundaries. As an example, consider generating a Poisson-disk-like point set on the surface of a sphere. The natural distance function is the length along arcs of great circles, and the boundary of a point is a small circle (the circle resulting from the intersection of a plane with the sphere). Available portions of this boundary can still be represented as angular ranges and determined by small circle intersection. Sampling can still be done in linear time by dividing the domain into a grid; for optimal performance, a grid should be used with roughly equi-area cells.

5.2 Generating Infinite Point Sets

It is not always easy to predict the number of required points for an application. In these cases, it is convenient to be able to generate points near an arbitrary position in the plane. This ability also permits a Poisson-disk texture basis functions as in Lagae and Dutré’s method [2005], where values such as a unique nearest-point identifier or distance to the closest point are returned for any 2D location. Existing methods use precomputed tiles to solve this problem.

We have successfully applied scalloped sectors to the problem of sampling from infinite point sets. By judicious seeding of a pseudorandom number generator, we are able to generate points in arbitrary subsets of the plane while guaranteeing that the resulting point sets will not violate the Poisson-disk property when placed next to neighboring point sets generated by our technique. This seems like a promising way to compute infinite point sets without tiling artifacts; we have not yet implemented any applications that leverage this capability.

5.3 Importance Sampling

Given an importance function over the domain, it is possible to derive a non-Euclidean distance metric for which our approach would yield a point set whose density was locally proportional to that function. These point sets should have local blue noise properties as in Ostromoukhov et al. [2004] and Kopf et al [2006].

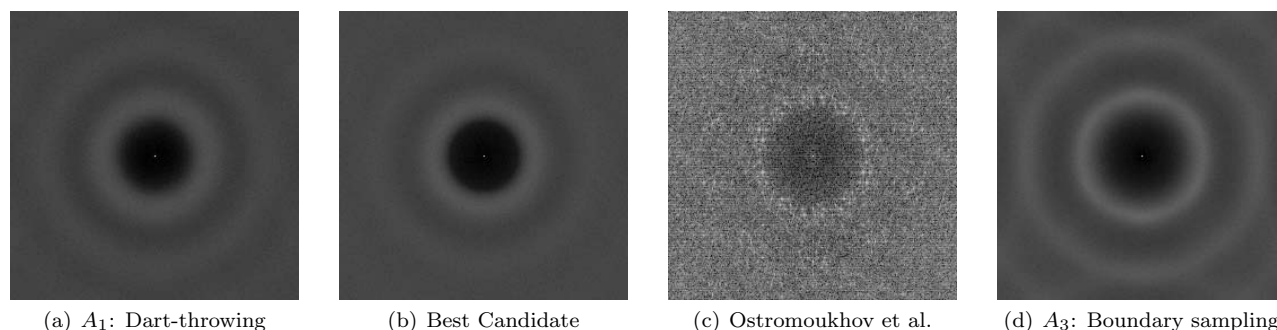


Figure 5: Averaged periodograms for several sampling methods. Boundary sampling generates the best blue noise spectrum due to its extremely regular and dense sampling of the plane. Ostromoukhov et al.’s results are noisier because their technique does not involve randomness, so averaging multiple runs does not provide smooth periodograms.

In general, this metric will not have a closed analytic form, and therefore the boundary cannot be directly determined. Nonetheless, it would be possible to solve for a piecewise linear (or higher order, if necessary) approximation of the distance metric in the vicinity of a point and use that to represent the local boundary. By applying this technique and the method in 5.1 we hope to be able to extend our technique to support efficient importance sampling.

We intend to apply these results combined with those in the previous section in order to be able to place objects with varying radii across unbounded regions. We expect this to be valuable for generating complex terrain and environments.

References

- COHEN, M. F., SHADE, J., HILLER, S., AND DEUSSEN, O. 2003. Wang tiles for image and texture generation. *ACM Transactions on Graphics* 22, 3, 287–294.
- COOK, R. L. 1986. Stochastic sampling in computer graphics. *ACM Transactions on Graphics* 5, 1, 51–72.
- DIPPÉ, M. A. Z., AND WOLD, E. H. 1985. Antialiasing through stochastic sampling. In *Computer Graphics (Proceedings of SIGGRAPH 85)*, ACM Press, New York, NY, USA, 69–78.
- DUNBAR, D., AND HUMPHREYS, G. 2006. Using scalloped sectors to generate poisson-disk sampling patterns. Tech. Rep. CS-2006-08, University of Virginia.
- HILLER, S., DEUSSEN, O., AND KAUFMANN, A. 2001. Tiled blue noise samples. In *VMV ’01: Proceedings of the Vision Modeling and Visualization Conference 2001*, Aka GmbH, 265–272.
- JONES, T. 2006. Efficient generation of poisson-disk sampling patterns. *Journal of Graphics Tools, to appear*.
- KOLLIG, T., AND KELLER, A. 2002. Efficient multidimensional sampling. *Computer Graphics Forum* 21, 3, 557–563.
- KOLLIG, T., AND KELLER, A. 2003. Efficient illumination by high dynamic range images. *Rendering Techniques*, 45–51.
- KOPF, J., COHEN-OR, D., DEUSSEN, O., AND LISCHINSKI, D. 2006. Recursive wang tiles for real-time blue noise. *ACM Transactions on Graphics* 25, 3.
- LAGAE, A., AND DUTRÉ, P. 2005. A procedural object distribution function. *ACM Transactions on Graphics* 24, 4, 1442–1461.
- MCCOOL, M., AND FIUME, E. 1992. Hierarchical poisson disk sampling distributions. In *Proceedings of the conference on Graphics interface ’92*, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 94–105.
- MITCHELL, D. P. 1991. Spectrally optimal sampling for distribution ray tracing. In *Computer Graphics (Proceedings of ACM SIGGRAPH 91)*, ACM Press, New York, NY, USA, 157–164.
- OSTROMOUKHOV, V., DONOHUE, C., AND JODOIN, P.-M. 2004. Fast hierarchical importance sampling with blue noise properties. *ACM Transactions on Graphics* 23, 3, 488–495.
- SHIRLEY, P. 1991. Discrepancy as a quality measure for sample distributions. In *Proceedings of Eurographics*, 183–194.
- ULICHNEY, R. A. 1988. Dithering with blue noise. In *Proc. of the IEEE* 76, 56–79.
- YELLOTT, J. I. 1983. Spectral consequences of photoreceptor sampling in the rhesus retina. *Science* 221, 382–385.